

Blazor Integration

This guide covers how to integrate ReportViewerCore with Blazor applications (Blazor Server, Blazor WebAssembly, and Blazor Hybrid).

Important Considerations

Blazor WebAssembly Limitations

ReportViewerCore cannot run directly in Blazor WebAssembly because:

- Report processing requires server-side .NET runtime
- RDLC compilation uses Roslyn compiler (not available in browser)
- Native dependencies (fonts, rendering) require full .NET runtime

Solution: Use a backend API for report generation (see patterns below)

Blazor Server & Hybrid

These modes can use ReportViewerCore directly since they run on the server/.NET runtime.

Installation

```
# Blazor Server / Blazor Hybrid
dotnet add package Microsoft.Reporting.NETCore

# For Blazor WebAssembly, install in your API project
cd YourApp.API
dotnet add package Microsoft.Reporting.NETCore
```

Architecture Patterns

Pattern 1: Backend API + Blazor (Recommended for WASM)

API Controller

```
// API Project - Controllers/ReportsController.cs
[ApiController]
[Route("api/[controller]")]
public class ReportsController : ControllerBase
{
    private readonly IReportGeneratorService _reportService;

    public ReportsController(IReportGeneratorService reportService)
    {
        _reportService = reportService;
    }
}
```

```

    }

    [HttpPost("generate")]
    public IActionResult GenerateReport([FromBody] ReportRequest request)
    {
        var report = _reportService.GenerateReport(
            request.ReportName,
            request.Data,
            request.Parameters,
            request.Format
        );

        var contentType = GetContentType(request.Format);
        var fileName = $"{request.ReportName}.{GetExtension(request.Format)}";

        return File(report, contentType, fileName);
    }

    [HttpGet("invoice/{id}/{format}")]
    public IActionResult GetInvoice(int id, string format = "pdf")
    {
        var invoiceData = GetInvoiceData(id);
        var report = GenerateInvoiceReport(invoiceData, format.ToUpper());

        var contentType = GetContentType(format);
        return File(report, contentType, $"Invoice_{id}.{format}");
    }
}

```

Blazor Component

```

@* Pages/Reports/InvoiceViewer.razor *@
@page "/reports/invoice/{InvoiceId:int}"
@inject HttpClient Http
@inject IJSRuntime JS

<h3>Invoice Report</h3>

<div class="report-controls">
    <button class="btn btn-primary" @onclick="() => DownloadReport(\"pdf\")">
        Download PDF
    </button>
    <button class="btn btn-success" @onclick="() => DownloadReport(\"excel\")">
        Download Excel
    </button>
    <button class="btn btn-info" @onclick="PreviewReport">
        Preview
    </button>
</div>

```

```

@if (isLoading)
{
    <div class="spinner-border" role="status">
        <span class="sr-only">Loading...</span>
    </div>
}

@if (!string.IsNullOrEmpty(errorMessage))
{
    <div class="alert alert-danger">@errorMessage</div>
}

@if (showPreview && htmlContent != null)
{
    <div class="report-preview">
        @((MarkupString)htmlContent)
    </div>
}

@code {
    [Parameter]
    public int InvoiceId { get; set; }

    private bool isLoading = false;
    private string errorMessage;
    private bool showPreview = false;
    private string htmlContent;

    private async Task DownloadReport(string format)
    {
        isLoading = true;
        errorMessage = null;

        try
        {
            var response = await
Http.GetAsync($"api/reports/invoice/{InvoiceId}/{format}");

            if (response.IsSuccessStatusCode)
            {
                var content = await response.Content.ReadAsByteArrayAsync();
                var fileName = $"Invoice_{InvoiceId}.{format}";

                await JS.InvokeVoidAsync("downloadFile", fileName,
                    Convert.ToBase64String(content),
                    response.Content.Headers.ContentType.ToString());
            }
            else
            {
                errorMessage = "Failed to generate report";
            }
        }
        catch (Exception ex)

```

```

        {
            errorMessage = $"Error: {ex.Message}";
        }
        finally
        {
            isLoading = false;
        }
    }

private async Task PreviewReport()
{
    isLoading = true;
    errorMessage = null;
    showPreview = false;

    try
    {
        htmlContent = await
Http.GetStringAsync($"api/reports/invoice/{InvoiceId}/html5");
        showPreview = true;
    }
    catch (Exception ex)
    {
        errorMessage = $"Error: {ex.Message}";
    }
    finally
    {
        isLoading = false;
    }
}
}

```

JavaScript Interop for Downloads

```

<!-- wwwroot/index.html or _Host.cshtml -->
<script>
    window.downloadFile = function(filename, content, contentType) {
        // Convert base64 to blob
        const byteCharacters = atob(content);
        const byteNumbers = new Array(byteCharacters.length);
        for (let i = 0; i < byteCharacters.length; i++) {
            byteNumbers[i] = byteCharacters.charCodeAt(i);
        }
        const byteArray = new Uint8Array(byteNumbers);
        const blob = new Blob([byteArray], { type: contentType });

        // Create download link
        const url = window.URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
    }

```

```
link.download = filename;
document.body.appendChild(link);
link.click();
document.body.removeChild(link);
window.URL.revokeObjectURL(url);
};
</script>
```

Pattern 2: Blazor Server Direct Integration

```
// Services/ReportService.cs
public class BlazorReportService
{
    private readonly IWebHostEnvironment _environment;

    public BlazorReportService(IWebHostEnvironment environment)
    {
        _environment = environment;
    }

    public byte[] GenerateReport<T>(
        string reportName,
        IEnumerable<T> data,
        Dictionary<string, string> parameters,
        string format = "PDF")
    {
        using var report = new LocalReport();

        var reportPath = Path.Combine(
            _environment.ContentRootPath,
            "Reports",
            $"{reportName}.rdlc"
        );

        using var stream = File.OpenRead(reportPath);
        report.LoadReportDefinition(stream);

        report.DataSources.Add(new ReportDataSource("DataSet1", data));

        if (parameters != null)
        {
            var reportParams = parameters
                .Select(p => new ReportParameter(p.Key, p.Value))
                .ToArray();
            report.SetParameters(reportParams);
        }

        return report.Render(format);
    }
}
```

```

@* Blazor Server Component *@
@page "/reports/sales"
@inject BlazorReportService ReportService
@inject IJSRuntime JS

<h3>Sales Report</h3>

<EditForm Model="@reportRequest" OnValidSubmit="GenerateReport">
    <div class="form-group">
        <label>Start Date:</label>
        <InputDate @bind-Value="reportRequest.StartDate" class="form-control"
    />
    </div>
    <div class="form-group">
        <label>End Date:</label>
        <InputDate @bind-Value="reportRequest.EndDate" class="form-control" />
    </div>
    <div class="form-group">
        <label>Format:</label>
        <InputSelect @bind-Value="reportRequest.Format" class="form-control">
            <option value="PDF">PDF</option>
            <option value="EXCELOPENXML">Excel</option>
            <option value="WORDOPENXML">Word</option>
        </InputSelect>
    </div>
    <button type="submit" class="btn btn-primary">Generate Report</button>
</EditForm>

@code {
    private ReportRequest reportRequest = new() { Format = "PDF" };

    private async Task GenerateReport()
    {
        var data = GetSalesData(reportRequest.StartDate,
reportRequest.EndDate);

        var parameters = new Dictionary<string, string>
        {
            ["StartDate"] = reportRequest.StartDate.ToString("MM/dd/yyyy"),
            ["EndDate"] = reportRequest.EndDate.ToString("MM/dd/yyyy"),
            ["GeneratedDate"] = DateTime.Now.ToString("MM/dd/yyyy")
        };

        var reportBytes = ReportService.GenerateReport(
            "SalesReport",
            data,
            parameters,
            reportRequest.Format
        );
    }
}

```

```

        var fileName = $"SalesReport_{DateTime.Now:yyyyMMdd}.
        {GetExtension(reportRequest.Format)}";

        await JS.InvokeVoidAsync("downloadFile",
            fileName,
            Convert.ToBase64String(reportBytes),
            GetContentType(reportRequest.Format));
    }

    private string GetExtension(string format) => format.ToUpper() switch
    {
        "EXCELOPENXML" => "xlsx",
        "WORDOPENXML" => "docx",
        _ => "pdf"
    };

    private string GetContentType(string format) => format.ToUpper() switch
    {
        "EXCELOPENXML" => "application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet",
        "WORDOPENXML" => "application/vnd.openxmlformats-
officedocument.wordprocessingml.document",
        _ => "application/pdf"
    };

    private class ReportRequest
    {
        public DateTime StartDate { get; set; } = DateTime.Now.AddMonths(-1);
        public DateTime EndDate { get; set; } = DateTime.Now;
        public string Format { get; set; }
    }
}

```

Pattern 3: Report Preview Component

Create a reusable component for report previews:

```

@* Components/ReportViewer.razor *@
@inject HttpClient Http

<div class="report-viewer">
    @if (IsLoading)
    {
        <div class="loading-overlay">
            <div class="spinner"></div>
            <p>Generating report...</p>
        </div>
    }

    @if (!string.IsNullOrEmpty(ErrorMessage))

```

```

    {
        <div class="alert alert-danger">
            @ErrorMessage
        </div>
    }

    @if (ReportFormat == "HTML5" && HtmlContent != null)
    {
        <div class="report-content">
            @((MarkupString)HtmlContent)
        </div>
    }
    else if (ReportFormat == "PDF" && PdfBase64 != null)
    {
        <iframe src="@($"data:application/pdf;base64,{PdfBase64}")"
            style="width: 100%; height: 800px;">
        </iframe>
    }
</div>

@code {
    [Parameter]
    public string ReportEndpoint { get; set; }

    [Parameter]
    public string ReportFormat { get; set; } = "PDF";

    [Parameter]
    public EventCallback OnReportGenerated { get; set; }

    public bool IsLoading { get; private set; }
    public string ErrorMessage { get; private set; }
    public string HtmlContent { get; private set; }
    public string PdfBase64 { get; private set; }

    protected override async Task OnParametersSetAsync()
    {
        await LoadReport();
    }

    public async Task LoadReport()
    {
        IsLoading = true;
        ErrorMessage = null;
        HtmlContent = null;
        PdfBase64 = null;

        try
        {
            if (ReportFormat.ToUpper() == "HTML5")
            {
                HtmlContent = await Http.GetStringAsync(ReportEndpoint);
            }
        }
    }
}

```

```

        else if (ReportFormat.ToUpper() == "PDF")
        {
            var pdfBytes = await Http.GetByteArrayAsync(ReportEndpoint);
            PdfBase64 = Convert.ToBase64String(pdfBytes);
        }

        await OnReportGenerated.InvokeAsync();
    }
    catch (Exception ex)
    {
        ErrorMessage = $"Failed to load report: {ex.Message}";
    }
    finally
    {
        IsLoading = false;
    }
}

public async Task Refresh()
{
    await LoadReport();
}
}

```

Usage:

```

<ReportViewer ReportEndpoint="@($\"api/reports/invoice/{InvoiceId}/html5\")"
               ReportFormat="HTML5"
               OnReportGenerated="OnReportLoaded" />

@code {
    [Parameter]
    public int InvoiceId { get; set; }

    private void OnReportLoaded()
    {
        // Handle report loaded event
        Console.WriteLine("Report loaded successfully");
    }
}

```

Advanced Scenarios

Dynamic Report Selection

```

@page "/reports/dynamic"
[inject HttpClient Http

```

```

<h3>Generate Report</h3>

<div class="form-group">
    <label>Select Report:</label>
    <select @bind="selectedReport" class="form-control">
        <option value="">-- Select Report --</option>
        <option value="sales">Sales Report</option>
        <option value="inventory">Inventory Report</option>
        <option value="customers">Customer Report</option>
    </select>
</div>

@if (!string.IsNullOrEmpty(selectedReport))
{
    <DynamicComponent Type="@GetReportComponent()" />
}

@code {
    private string selectedReport;

    private Type GetReportComponent() => selectedReport switch
    {
        "sales" => typeof(SalesReportComponent),
        "inventory" => typeof(InventoryReportComponent),
        "customers" => typeof(CustomerReportComponent),
        _ => null
    };
}

```

Batch Report Generation

```

@page "/reports/batch"
@Inject HttpClient Http

<h3>Batch Report Generation</h3>

<button @onclick="GenerateBatchReports" class="btn btn-primary">
    Generate All Reports
</button>

<div class="progress mt-3" style="display: @(isGenerating ? "block" : "none")">
    <div class="progress-bar" style="width: @progress%">
        @progress%
    </div>
</div>

<ul class="list-group mt-3">
    @foreach (var result in results)
    {
        <li class="list-group-item">

```

```

        @result.ReportName:
        @if (result.Success)
        {
            <span class="badge badge-success">Success</span>
            <a href="#" @onclick="() =>
DownloadReport(result)">Download</a>
        }
        else
        {
            <span class="badge badge-danger">Failed</span>
        }
    </li>
}
</ul>

@code {
    private bool isGenerating = false;
    private int progress = 0;
    private List<ReportResult> results = new();

    private async Task GenerateBatchReports()
    {
        isGenerating = true;
        progress = 0;
        results.Clear();

        var reports = new[] { "Sales", "Inventory", "Customers", "Orders" };
        int completed = 0;

        foreach (var reportName in reports)
        {
            try
            {
                var response = await
Http.GetAsync($"api/reports/{reportName}/pdf");
                var bytes = await response.Content.ReadAsByteArrayAsync();

                results.Add(new ReportResult
                {
                    ReportName = reportName,
                    Success = response.IsSuccessStatusCode,
                    Data = bytes
                });
            }
            catch
            {
                results.Add(new ReportResult
                {
                    ReportName = reportName,
                    Success = false
                });
            }
        }
    }
}

```

```

        completed++;
        progress = (completed * 100) / reports.Length;
        StateHasChanged();
    }

    isGenerating = false;
}

private class ReportResult
{
    public string ReportName { get; set; }
    public bool Success { get; set; }
    public byte[] Data { get; set; }
}
}

```

Service Registration

Blazor Server

```

// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddServerSideBlazor();
builder.Services.AddScoped<BlazorReportService>();
builder.Services.AddScoped<IReportGeneratorService, ReportGeneratorService>();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapBlazorHub();
app.MapFallbackToPage("/_Host");

app.Run();

```

Blazor WebAssembly

```

// Program.cs (Client)
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");

builder.Services.AddScoped(sp =>
    new HttpClient { BaseAddress = new Uri(builder.HostEnvironment.BaseAddress)
});

```

```
await builder.Build().RunAsync();
```

```
// Program.cs (Server/API)
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddScoped<IReportGeneratorService, ReportGeneratorService>();

// Enable CORS for Blazor WASM
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowBlazorWasm", policy =>
    {
        policy.WithOrigins("https://localhost:5001")
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

var app = builder.Build();

app.UseCors("AllowBlazorWasm");
app.MapControllers();

app.Run();
```

Best Practices

1. **Always use backend API for WASM:** Report generation must happen server-side
2. **Cache report definitions:** Load RDLC files once and reuse
3. **Use progress indicators:** Report generation can take time
4. **Handle errors gracefully:** Show user-friendly error messages
5. **Implement cancellation:** Allow users to cancel long-running reports
6. **Use streaming for large reports:** Avoid loading entire report in memory
7. **Secure endpoints:** Implement authentication and authorization
8. **Optimize data queries:** Only fetch required data for reports

Troubleshooting

Report not generating in Blazor WASM

Cause: ReportViewerCore requires server-side .NET runtime

Solution: Use backend API pattern (Pattern 1)

JavaScript interop errors

Ensure: JavaScript functions are defined before Blazor initializes

```
<script src="_framework/blazor.webassembly.js"></script>
<script src="js/report-download.js"></script>
```

Large reports timing out

Solution: Increase timeout and use streaming

```
builder.Services.AddHttpClient("reports", client =>
{
    client.Timeout = TimeSpan.FromMinutes(5);
});
```

Next Steps

- [MAUI Integration](#) - Use reports in mobile apps
- [API vs Standalone Mode](#) - Different deployment patterns
- [Advanced Topics](#) - Complex scenarios and optimizations